ДИСЦИПЛИНА «DEVOPS-ИНСТРУМЕНТЫ»

Лабораторная работа 1
Часть 2

Разбор запроса и статистика проекта

Неделя 2. Разделы 5-8 и полный итоговый чек-лист лабораторной работы 1.

Сведения	Значение
Выполнила	Есімханова Аружан
Группа	25-ИСТК-2о
Дата отчёта	22.10.2026
Проект	DemoApi - сервис проектов и задач
Режим хранения	InMemory

ГРАНИЦЫ ПОДТВЕРЖДЕНИЯ

В отчёте разделены фактические HTTP-ответы и результаты CLI-проверок, анализ исходного кода и действия, требующие ручной проверки в Postman / Visual Studio. Технические результаты подтверждены сохранёнными данными; полный комплект обязательных UI-доказательств ещё не собран.

Публичный репозиторий проекта

Дата и время проверок приводятся в часовом поясе UTC+05:00. Даты сроков задач и граница просрочки определяются по UTC.

1. Повторная проверка первой части

Итоговый чек-лист включает результаты первой недели. Коллекция demo-api сохранена в JSON; её baseUrl и фактическая проверка используют http://localhost:8088. Учебный порт 8080 занят Oracle, поэтому текущая версия согласованно использует 8088. Скриншот графического Postman пока не получен.

Успешный сценарий	Ожидание и фактический ответ
GET /api/projects	200; исходный список []
POST /api/projects	201; id=1; Location /api/projects/1
POST /api/tasks	201; id=1; projectId=1; status=Todo; Location /api/tasks/1
GET /api/tasks?projectId=1	200; одна созданная задача
PATCH /api/tasks/1/status	200; Todo -> InProgress
GET /api/projects/1	200; созданный проект

Проверка	Код	Фактические данные ответа
POST /api/projects, пустое name	400	errors.Name: "The Name field is required."
Повторное имя проекта	409	Проект «lab01-20261021T194046965Z-all-7024bbae-part1» уже существует
Новая Todo-задача сразу в Done	409	Нельзя перевести задачу 2 из Todo в Done
GET /api/tasks/2147483647	404	title: "Not Found"; status: 404

errors описывает ошибки полей модели запроса. **detail** объясняет конфликт состояния. Валидация name выполняется автоматически ASP.NET Core, а текст обоих ответов 409 задаёт контроллер. Для проверки отсутствующей задачи выбран int.MaxValue: на повторно используемом стенде он надёжнее фиксированного примера 999.

Данные в памяти

До завершения процесса: HTTP 200, 4 проекта (22.10.2026 00:34:34). После нового запуска: HTTP 200, тело [] (22.10.2026 00:40:46). Это фактическая проверка перезапуска процесса; последовательность клавиш Shift+F5 / F5 в Visual Studio ею не подтверждается.

InMemoryStore общий для запросов одного процесса. Завершение процесса уничтожает списки, а новый запуск создаёт пустое хранилище; сохранение в файл или БД в режиме InMemory отсутствует.

2. Две цепочки обработки запроса

АНАЛИЗ КОДА + ФАКТИЧЕСКИЙ HTTP

Цепочки ниже восстановлены чтением актуальных исходников и согласуются с HTTP-ответами. Прохождение F9 / F10 / F11, значения окна Locals и стек Call Stack в Visual Studio вручную не проверены. Текст не выдаётся за результат работы отладчика.

2.1. Запрещённый переход Todo -> Done

```
TasksController.ChangeStatus
-> TaskService.GetAsync
-> InMemoryTaskRepository.GetAsync
-> возврат найденной Todo-задачи
-> StatusRules.CanTransition(Todo, Done): false
-> return Problem(..., statusCode: 409)
```

Код 409 выбирается в TasksController.ChangeStatus в ветке !StatusRules.CanTransition(task.Status, status). Фактический запрос PATCH /api/tasks/2/status с телом {"status":"Done"} вернул 409; detail: «Нельзя перевести задачу 2 из Todo в Done».

TaskService.ChangeStatusAsync и InMemoryTaskRepository.SetStatusAsync в этой ветке не вызываются. По исходнику статус остаётся Todo. Возврат кода 409 также проверяется интеграционным тестом запрещённого перехода.

2.2. Разрешённый переход Todo -> InProgress

```
TasksController.ChangeStatus
-> TaskService.GetAsync
-> InMemoryTaskRepository.GetAsync
-> StatusRules.CanTransition(Todo, InProgress): true
-> TaskService.ChangeStatusAsync
-> InMemoryTaskRepository.SetStatusAsync
-> InMemoryTaskRepository.Replace
-> return updated: HTTP 200
```

Replace заменяет запись копией с новым Status и UpdatedAt внутри lock(store.Sync). Возвращённая задача передаётся через сервис контроллеру; return updated формирует 200. Если обновление вернуло null, предшествующая ветка возвращает 404. Фактический PATCH задачи 1 с InProgress вернул 200 и status=InProgress.

Для ручной проверки нужно создать одну новую Todo-задачу, сначала отправить Done, затем InProgress, сохранив Locals и Call Stack для обоих запросов. Сохранённый общий HTTP-прогон проверял эти переходы на двух разных задачах, поэтому не подменяет требуемый опыт отладки на одной задаче.

3. Ответы на шесть вопросов по коду

1. Где задан маршрут api/tasks/{id}/status?

В Controllers/TasksController.cs: [Route("api/tasks")] задаёт префикс класса, а [HttpPatch("{id:int}/status")] над ChangeStatus добавляет идентификатор, /status и метод PATCH. Ограничение :int допускает целочисленный id.

2. Кто возвращает 400 при пустом title?

ASP.NET Core с [ApiController] проверяет модель до вызова действия. У CreateTaskRequest.Title и UpdateTaskRequest.Title установлены [Required, StringLength(200)]. Пустое название приводит к 400 со списком ошибок полей в errors.

3. Где правило переходов и как проверить его без HTTP?

В Domain/StatusRules.cs: словарь Allowed и метод CanTransition. Допустимы соседние переходы Todo <-> InProgress <-> Done и сохранение текущего статуса; непосредственный Todo <-> Done запрещён. Unit/StatusRulesTests.cs вызывает правило напрямую с семью наборами данных, без HTTP и БД.

4. Где выбирается 404 и откуда detail у 409?

Контроллер задаёт HTTP-код: null после чтения или обновления приводит к NotFound(). Для конфликта статуса TasksController.ChangeStatus передаёт пояснение в Problem(..., 409); для дубликата имени это делает ProjectsController.Create. Аргумент detail превращается в одноимённое поле JSON.

5. Где выбирается хранилище и что используется?

В Program.cs значение builder.Configuration["Storage"] сравнивается с Postgres. При совпадении регистрируются SQL-репозитории, иначе InMemoryStore и InMemory-реализации. В данном запуске использовано InMemory; его поведение подтверждается API и очисткой данных после перезапуска. Окружение может переопределять JSON-настройки.

6. Почему TaskService не меняется при PostgreSQL?

TaskService принимает ITaskRepository и IProjectRepository. Оба способа хранения реализуют одинаковые интерфейсы; при изменении настройки контейнер зависимостей подставляет другой класс. Сервис продолжает вызывать те же методы и не содержит SQL.

ПРОВЕРЯЕМЫЕ ГРАНИЦЫ

Ответы основаны на исходном коде включённой версии. Работоспособность PostgreSQL-реализаций во внешней БД в лабораторной 1 не проверялась.

Самопроверка. Шесть вопросов лекции

Эти вопросы относятся к лекции «Лаба 1,3.pdf» и дополняют шесть практических вопросов предыдущего раздела.

1. Frontend, backend и клиенты сервиса

Frontend показывает интерфейс пользователю, backend выполняет логику и работает с данными. DemoApi предоставляет JSON API без собственных пользовательских страниц. К нему могут обращаться сайт, мобильное приложение, другие сервисы, Postman, Swagger и клиенты командной строки.

2. Чем PUT отличается от PATCH и почему создание возвращает 201?

PUT заменяет редактируемые поля ресурса целиком: у задачи это название, описание и срок. PATCH изменяет отдельную часть - например статус. Ответ 201 Created означает появление нового ресурса, его адрес передаётся в Location; код 200 не имеет специального смысла создания.

3. Почему POST принимает CreateTaskRequest, а не TaskItem?

Модель запроса содержит только разрешённые поля: projectId, title, description и dueDate. Идентификатор, исходный статус Todo и даты создания / обновления назначает сервер. При приёме TaskItem целиком клиент мог бы пытаться подменить эти значения и обойти правила.

4. Что будет, если PATCH /status не передаст status?

ChangeStatusRequest.Status имеет nullable-тип и атрибут [Required]. Отсутствующее значение не превращается в Todo по умолчанию: [ApiController] автоматически возвращает 400 с ошибкой модели до выполнения действия контроллера.

5. Как выбирается хранилище без изменения TaskService?

Program.cs читает Storage и регистрирует в контейнере DI либо InMemory-, либо SQL-реализации ITaskRepository и IProjectRepository. TaskService вызывает интерфейсы, поэтому переключение конфигурации не требует изменения его кода.

6. Зачем UseHttpMetrics стоит раньше UseExceptionHandler?

Middleware метрик оборачивает обработчик ошибок и видит окончательный HTTP-код ответа. Если обработчик преобразовал исключение в 500, именно этот код учитывается в HTTP-метриках приложения.

4. Реализация статистики проекта

Добавлена операция **GET /api/projects/{id}/stats**. Она возвращает количество задач указанного проекта по статусам и срокам. Перед подсчётом контроллер проверяет существование проекта: неизвестный id получает 404, а существующий пустой проект - 200 с нулевыми счётчиками.

Компонент	Изменение
Domain/Project.cs	ProjectStats: ProjectId, Total, Todo, InProgress, Done, Overdue; XML-описания полей
Data/ProjectRepository.cs	GetStatsAsync в интерфейсе и SQL-реализации; параметризованные COUNT FILTER
Data/InMemory/InMemoryProjectRepository.cs	LINQ-подсчёты только своего проекта внутри lock(store.Sync)
Services/ProjectService.cs	GetStatsAsync делегирует репозиторию
Controllers/ProjectsController.cs	Маршрут /{id:int}/stats; существование проекта; 200/404; XML-документация
DemoApi.http и ProjectsApiTests.cs	Пример HTTP-запроса и два новых интеграционных теста

Правило просрочки и часовой пояс

```
dueDate < DateOnly.FromDateTime(DateTime.UtcNow)
&& status != TaskItemStatus.Done
```

Дата сегодня не просрочена; отсутствие срока и завершённая задача не увеличивают overdue. Для обеих реализаций граница дня берётся по UTC. В SQL вместо зависящего от настроек сессии current_date передаётся параметр даты UTC. Это сохраняет одинаковый смысл результата для InMemory и PostgreSQL.

В SQL projectId и дата передаются параметрами, а не склеиваются с текстом запроса. COUNT возвращает bigint; чтение выполняется через GetInt64 с checked-преобразованием в int. Новые зависимости и архитектурные слои не добавлены.

5. Фактический ответ статистики

Для независимого проекта id=2 созданы три задачи: Todo без срока, InProgress и Todo со сроком позавчера по UTC. Срок последней назначен через PUT после обычного создания. Так демонстрация overdue остаётся совместимой с дополнительным запретом прошлой даты только при POST.

Задача	Статус	Срок / вклад в overdue
Todo without a due date	Todo	null / 0
Task in progress	InProgress	null / 0
Overdue Todo task	Todo	2026-10-19 / 1; дата запроса UTC 2026-10-21

Получено от сервера

```
GET /api/projects/2/stats
HTTP 200 OK
{
"projectId": 2,
"total": 3,
"todo": 2,
"inProgress": 1,
"done": 0,
"overdue": 1
}
```

Поле	Смысл	Ожид.	Факт
projectId	Идентификатор проекта	2	2
total	Все задачи проекта	3	3
todo	Задачи Todo	2	2
inProgress	Задачи InProgress	1	1
done	Завершённые задачи	0	0
overdue	Просроченные незавершённые	1	1

Все шесть значений совпали. GET /api/projects/2147483647/stats возвратил 404. Время HTTP-прогона: 22.10.2026 00:40:46 (UTC+05:00). Полные запросы, ответы и проверки сохранены в приложенных доказательствах.

Демонстрация. Реальный запрос в Swagger

Через браузерный Swagger выполнен GET /api/projects/2/stats: Try it out, затем Execute. На подлинном снимке видны Request URL, Server response 200 и JSON со значениями 3 / 2 / 1 / 0 / 1.

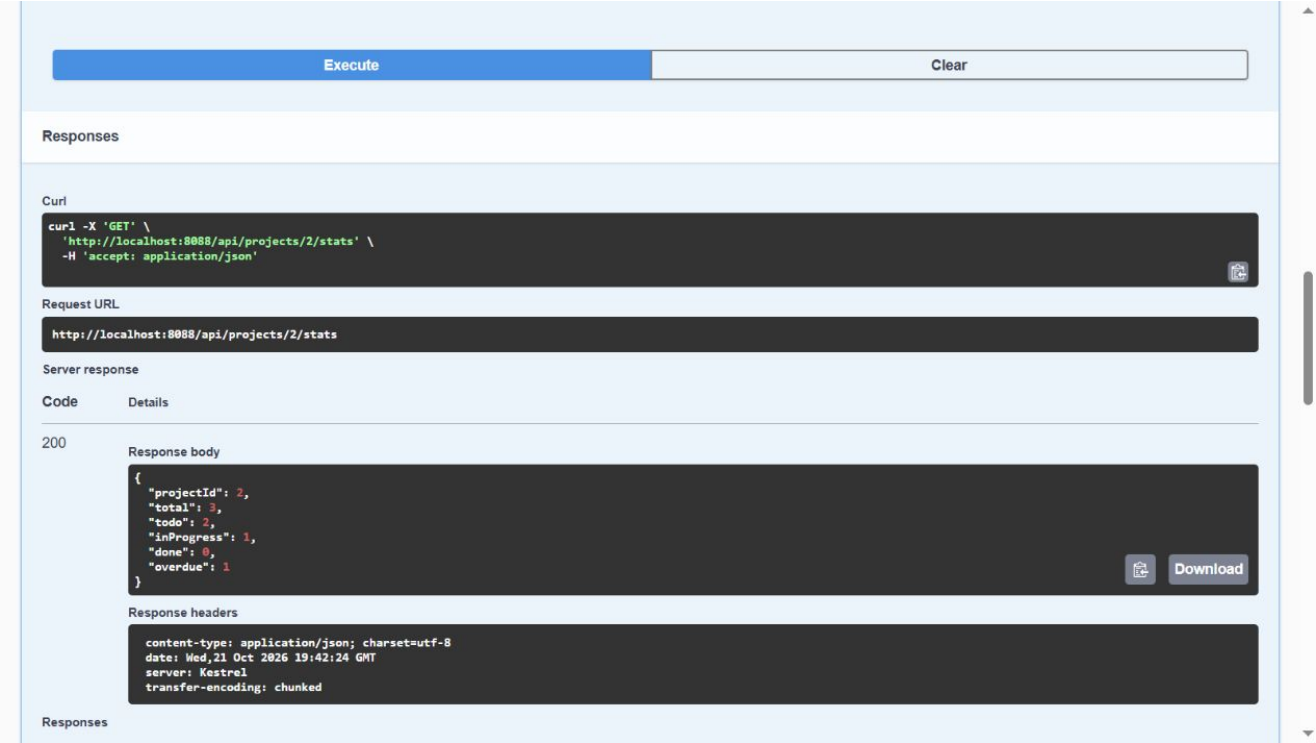


Рисунок 1. Swagger UI: фактический ответ сервера на запрос статистики проекта 2. Заголовок Date: 21.10.2026 19:42:24 GMT, то есть 22.10.2026 00:42:24 UTC+05:00.

На снимке отображается результат выполненного запроса к localhost:8088. Это подтверждение работы операции через Swagger. Изображение не является скриншотом коллекции Postman, окна отладчика или Test Explorer и не закрывает соответствующие отдельные пункты сдачи.

Документация API

Операция статистики включена в OpenAPI/Swagger через маршрут контроллера и XML-комментарии. Поля ProjectStats документированы в исходнике. Swagger и /openapi/v1.json включены в окружении Development; для пользователя сервиса это технический HTTP-клиент, а не собственный frontend приложения.

6. Тесты и воспроизводимые проверки

Результаты ниже прочитаны из сохранённых сводок фактических TRX. Тестирование выполнено через CLI; скриншот Test Explorer не получен. Интеграционные тесты используют WebApplicationFactory и настоящий HTTP-конвейер приложения с хранилищем InMemory.

Этап	Всего / passed / failed	Время UTC+05:00
Исходное решение	22 / 22 / 0	22.10.2026 00:15:12
Обязательная статистика	24 / 24 / 0	22.10.2026 00:37:54
Postman CLI: обязательные папки	22 запроса; 67 assertions; 0 failed	22.10.2026, отдельный сохранённый запуск
Postman CLI: полная коллекция	24 запроса; 73 assertions; 0 failed	22.10.2026, отдельный сохранённый запуск
HTTP-скрипт: All	23 запроса; passed=true	22.10.2026 00:40:46

Два новых обязательных теста

```
ProjectsApiTests.Stats_ReturnsProjectCountsAndOverdueTasks
ProjectsApiTests.Stats_UnknownProject_ReturnsNotFound
```

Успешный тест проверяет счётчики и просрочку, а также пустой проект, срок сегодня, отсутствие срока, изоляцию задач другого проекта и исключение завершённой просроченной задачи. Второй тест проверяет 404 для неизвестного проекта.

Итоговые обязательные 24 случая состоят из 7 вариантов юнит-теста правил статуса и 17 интеграционных проверок. Ранее существовавший TasksApiTests.ChangeStatus_TodoToDone_ReturnsConflict проверяет запрет непосредственного Todo -> Done.

Версия обязательного результата

Тег lab01-required-24 / коммит f93b123 фиксирует состояние с 24 обязательными тестами до дополнительного правила. Архив исходников позволяет проверить этот результат отдельно от расширенной версии.

ЧТО ЭТИ ПРОВЕРКИ НЕ ПОДТВЕРЖДАЮТ

TRX и Postman CLI не доказывают наличие скриншотов Test Explorer или Postman, работу F9 / F10 / F11 и выполнение запросов через кнопку Send Request в Visual Studio. Исполнение SQL на PostgreSQL также не входит в эти результаты.

7. Дополнительное правило срока

Дополнительное задание отмечено в методичке звёздочкой: при создании задачи срок в прошлом должен приводить к 400. Расширенный набор проверен отдельно от сохранённого обязательного результата 24 теста.

Правило реализовано явно в TasksController.Create. Если DueDate меньше текущей даты UTC, контроллер добавляет ошибку DueDate в ModelState и возвращает ValidationProblem(ModelState), не вызывая создание задачи. Такая локальная проверка одного зависящего от времени условия не требует нового атрибута и не меняет разрешённый PUT.

Проверка	Ожидаемое поведение
Прошлая дата при POST	400; новая задача не создаётся
Срок сегодня	Допустим
Будущая дата	Допустима
Срок не задан	Допустим

Фактический результат расширенной проверки

```
Всего: 28
Выполнено успешно: 28
Ошибок: 0
Режим: InMemory
```

Время завершения по сохранённой сводке: 22.10.2026 00:39:59 (UTC+05:00). Это отдельный результат; он не подменяет зафиксированные ранее 24 обязательных теста.

В итоговом HTTP-прогоне POST с dueDate=2026-10-19 вернул 400. Фактическое errors.DueDate: «Срок новой задачи не может быть в прошлом (дата UTC).». Тест отклонения проверяет также отсутствие созданной записи; три дополнительных случая проверяют допустимые даты null, сегодня и завтра.

Почему просроченная статистика остаётся полезной

Задача может стать просроченной с течением времени, даже если при создании дата была допустима. Для воспроизводимого теста дата существующей задачи переводится в прошлое через PUT. Поэтому правило валидации POST и правило подсчёта overdue решают разные задачи и не должны блокировать друг друга.

Ручная проверка графических инструментов

Для закрытия требований нужно импортировать demo-api в Postman и сохранить настоящий скриншот коллекции; в Visual Studio пройти два запроса отладчиком с Locals / Call Stack; в Test Explorer показать все 24 теста обязательной версии. Эти действия и изображения ещё не подтверждены.

8. Итоговый чек-лист и вывод

Требование методички	Фактическое состояние
Коллекция demo-api в Postman	JSON и запуск Postman CLI подтверждены; скриншот UI отсутствует
Коды, errors и detail из раздела 3	Приведены дословные значения реальных ответов
Ответ о хранении и перезапуске	Объяснение дано; HTTP-снимки подтверждены
Две цепочки вызовов	Разобраны по коду + HTTP; ручная отладка и Locals / Call Stack не подтверждены
Шесть ответов по исходникам	Даны в разделе 3
Ответ GET /api/projects/{id}/stats	200; 3 / 2 / 1 / 0 / 1; неизвестный id - 404
Все 24 обязательных теста	24 passed, 0 failed через CLI; скрин Test Explorer ещё нужен
Код функции архивом или текстом	Отдельный lab01-source.zip и публичный репозиторий
Дополнительный запрет прошлой даты	28 passed, 0 failed отдельно

Вывод

Реализована статистика проекта с подсчётом статусов и просроченных незавершённых задач. Поведение InMemory подтверждено HTTP-проверками и обязательными 24 тестами; сохранены коллекция, результаты запросов и исходный код. SQL-реализация включена в проект, но её исполнение на PostgreSQL относится к дальнейшей проверке.

Полная сдача лабораторной пока не заявляется: остаются подлинный скриншот коллекции Postman, ручной проход двух запросов отладчиком и скриншот обязательных 24 тестов в Test Explorer. Настоящие UI-доказательства следует добавить к отчёту после выполнения этих действий.

Приложение. Исходники и воспроизведение

Материалы и версии

Основание: «Лаба 1,1.pdf» и «Лаба 1,3.pdf» - лекция; «Лаба 1,2.pdf» - практическая часть 1; «Лаба 1,4.pdf» - практическая часть 2. Условия на фотографиях уточняют состав контрольных точек. Вторая часть включает итоговый чек-лист первой.

Исходный код и документация
Обязательная версия: тег lab01-required-24, коммит f93b123

Архив исходного кода в репозитории

К отчётам прилагается отдельный архив **lab01-source.zip**: исходники src и tests, коллекция Postman, скрипт воспроизведения и сохранённые доказательства. Архив не содержит установленных зависимостей и результатов сборки.

Доказательство в архиве / репозитории	Содержание
evidence/lab01/20261021T194046965Z-all-7024bbae/ http-evidence.json	Запросы и ответы API; коды, заголовки Location, тела JSON и проверки
evidence/lab01/postman-required/console.txt	Локальный запуск Postman CLI: 22 запроса, 67 assertions, 0 failed
evidence/lab01/postman-final/console.txt	Полная коллекция Postman CLI: 24 запроса, 73 assertions, 0 failed
evidence/lab01/baseline/summary.json	Сводка исходных 22 тестов из TRX
evidence/lab01/required-24/summary.json	Сводка обязательных 24 тестов из TRX
evidence/lab01/restart/	Четыре проекта до перезапуска процесса; пустой список после

Сводки тестов очищены от служебных сведений о локальном пользователе и компьютере. Вывод Postman CLI подтверждает локальные запросы; публикация результатов в Postman Cloud не выполнялась.

Повторить обязательную проверку

```
git checkout lab01-required-24
dotnet test DemoApi.slnx
# При заранее запущенном сервисе:
pwsh -File scripts/verify-lab01.ps1 -Phase Part2 `
-BaseUrl http://localhost:8088 -KeepData
```

Для проверки в Postman запускается папка «Часть 2 - статистика проекта» после реализации /stats. Дополнительная папка запускается отдельно, только после включения соответствующего правила. Локальные пути компьютера и учётные данные в отчёт не включаются.